

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №10

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: " Поиск расстояний во взвешенном графе"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М. А.

Принял:

к.н., доцент, Юрова О. В.

к.т.н., Деев М. В.

Пенза 2025

Цель

Изучение поиска расстояния во взвешенном графе.

Теоретическая часть

Во взвешенном графе в отличие от не взвешенного каждое ребро имеет вес, отличный от нуля. Поэтому в матрице смежности взвешенного графа содержится информация не только о наличии ребра, но и о его весе.

Поиск расстояний между вершинами в таком графе также возможно построить используя процедуры обхода графа. Отличие от поиска расстояний в не взвешенном графе будет состоять в том, что при обновлении расстояния до вершины при ее посещении оно будет увеличиваться не на 1, а на величину веса ребра.

Таким образом, можно предложить следующую реализацию алгоритма обхода в ширину.

Псевдокод

Вход: G – матрица смежности графа, v – исходная вершина.

Выход: $DIST$ – вектор расстояний до всех вершин от исходной.

Алгоритм ПОШ

1.1. для всех i положим $DIST[i] = -1$ пометим как "не посещенную";

1.2. ВЫПОЛНЯТЬ BFS(v).

1.3 для всех i вывести $DIST[i]$ на экран;

Алгоритм BFS(v):

2.1. Создать пустую очередь $Q = \{\}$;

2.2. Поместить v в очередь $Q.push(v)$;

2.3. Обновить вектор расстояний $DIST[x] = 0$;

2.4. ПОКА $Q \neq \emptyset$ очередь не пуста ВЫПОЛНЯТЬ

2.5. $v = Q.front()$ установить текущую вершину;

2.6. Удалить первый элемент из очереди $Q.pop()$;

2.7. вывести на экран v ;

2.8. ДЛЯ $i = 1$ ДО $size_G$ ВЫПОЛНЯТЬ

2.9. ЕСЛИ $G(v,i) > 0$ И $DIST[i] = -1$

2.10. ТО

2.11. Поместить i в очередь $Q.push(i)$;

2.12. Обновить вектор расстояний $DIST[i] = DIST[v] + G(v,i);$

Лабораторное задание:

а. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного взвешенного графа G . Выведите матрицу на экран.

б. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.

Результат программы

```
E:\gitnew\laba10\laba10\Debug\laba10.exe
Input number of vertission : 5
0 9 0 0 5
9 0 0 0 0
0 0 0 0 7
0 0 0 0 8
5 0 7 8 0

Distance matrix:
0 1 2 2 1
1 0 3 3 2
2 3 0 2 1
2 3 2 0 1
1 2 1 1 0

Vector eccentristy:
2 3 3 3 2_
```

Рисунок 1 – поиск расстояния в графе

Листинг

Файл 1- laba10.cpp

```
#define _CRT_SECURE_NO_WARNINGS

#include "stdafx.h"
#include <iostream>
#include <stdio.h>
#include <conio.h>
#include <cstdlib>
#include <ctime>
#include <limits.h>
#include <queue>

using namespace std;

void printM(int** Matr, int numG) {
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", Matr[i][j]);
        }
        printf("\n");
    }
}

void BFSM(int** G, int numG, int** GD, int s) {
    queue<int> q;
    int v;
    int* dist = (int*)malloc(numG * sizeof(int));

    for (int i = 0; i < numG; i++) {
        dist[i] = INT_MAX;
    }

    dist[s] = 0;
    q.push(s);

    while (!q.empty()) {
        v = q.front();
        q.pop();

        for (int i = 0; i < numG; i++) {
            if (G[v][i] > 0 && dist[i] > dist[v] + G[v][i]) {
                q.push(i);
                dist[i] = dist[v] + 1;
            }
        }
    }

    for (int i = 0; i < numG; i++) {
        GD[s][i] = (dist[i] == INT_MAX ? -1 : dist[i]);
    }

    free(dist);
}

int main() {

    int** G;
    int** GD;
    int* ecc;
    int numG, current;

    printf("Input number of vertission : ");
    scanf("%d", &numG);
```

```

ecc = (int*)malloc(numG * sizeof(int));
G = (int**)malloc(numG * sizeof(int*));
GD = (int**)malloc(numG * sizeof(int*));

for (int i = 0; i < numG; i++) {
    G[i] = (int*)malloc(numG * sizeof(int));
    GD[i] = (int*)malloc(numG * sizeof(int));
}

for (int i = 0; i < numG; i++) {
    ecc[i] = -1;
    for (int j = i; j < numG; j++) {
        G[i][j] = G[j][i] = (i == j ? 0 : (rand() % 2) ? rand() % 11 : 0);
        GD[i][j] = GD[j][i] = 0;
    }
}

for (int i = 0; i < numG; i++) {
    for (int j = 0; j < numG; j++) {
        printf("%3d", G[i][j]);
    }

    printf("\n");
}

for (int i = 0; i < numG; i++) {
    BFS(D, G, numG, GD, i);
}

printf("\n\n");

printf("Distance matrix: \n");
printM(GD, numG);
printf("\n");
printf("Vector eccentricity: \n");
for (int i = 0; i < numG; i++) {
    for (int j = 0; j < numG; j++) {
        ecc[i] = (ecc[i] < GD[i][j] ? GD[i][j] : ecc[i]);
    }
    printf("%4d", ecc[i]);
}

for (int i = 0; i < numG; i++) {
    free(G[i]);
}

free(G);
free(ecc);

getchar();
getchar();

return 0;
}

```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №10. В процессе выполнения работы были использованы знания о поиске расстояния во взвешенном графе.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/MakarMn/LiOAvIZ>